

Lecture 4

Search

15-122: Principles of Imperative Computation (Spring 2025)
Frank Pfenning

One of the fundamental and recurring problems in computer science is to find elements in collections, such as elements in sets. An important algorithm for this problem is *binary search*. We use binary search for an integer in a sorted array to exemplify it. As a preliminary study in this lecture we analyze *linear search*, which is simpler, but not nearly as efficient. Still it is often used when the requirements for binary search are not satisfied, for example, when the elements we have to search are not arranged in a sorted array.

Additional Resources

- [Review slides](https://cs.cmu.edu/~15122/handouts/slides/review/04-linearsearch.pdf) (<https://cs.cmu.edu/~15122/handouts/slides/review/04-linearsearch.pdf>)
- [Code for this lecture](https://cs.cmu.edu/~15122/handouts/code/04-linearsearch.tgz) (<https://cs.cmu.edu/~15122/handouts/code/04-linearsearch.tgz>)

In term of our learning goals, we address the following:

Computational Thinking: Developing contracts (pre- and post-conditions, assertions, and loop invariants) that establish the safety and correctness of imperative programs.

Evaluating the use of *order* (sorted data) as a problem-solving tool.

Identifying the difference between *specification* and *implementation*.

Algorithms and Data Structures: Describing linear search.

Programming: We will practice *deliberate programming* together in lectures. Furthermore, identifying, describing, and effectively using short-circuiting Boolean operators will play an important role.

1 Linear Search in an Unsorted Array

If we are given an array of integers A without any further information and have to decide if an element x is in A , we just have to search through it, element by element. We return `true` as soon as we find an element that equals x , `false` if no such element can be found.

```

1 bool is_in(int x, int[] A, int lo, int hi)
2 //@requires 0 <= lo && lo <= hi && hi <= \length(A);
3 {
4   for (int i = lo; i < hi; i++)
5     //@loop_invariant lo <= i && i <= hi;
6     {
7       if (A[i] == x) return true;
8     }
9   return false;
10 }
```

We used the statement `i++` which is equivalent to `i = i+1` to step through the array, element by element.

The precondition is very common when working with arrays. We pass an array, and we also pass bounds — typically we will let lo be 0 and hi be the length of the array. The added flexibility of allowing lo and hi to take other values will be useful if we want to limit search to the first n elements of an array and do not care about the others. It will also be useful later to express invariants such as *x is not among the first k elements of A* , which we will write in code as `!is_in(x, A, 0, k)` and which we will write in mathematical notation as $x \notin A[0, k)$.

The loop invariant is also typical for loops over an array. We examine every element (i ranges from lo to $hi - 1$). But we will have $i = hi$ after the last iteration, so the loop invariant which is checked *just before the exit condition* must allow for this case.

Could we strengthen the loop invariant, or write a post-condition? We could try something like

```
//@loop_invariant !is_in(x, A, lo, i);
```

where `!b` is the negation of b . However, it is difficult to make sense of this use of recursion in a contract or loop invariant so we will avoid it.

This is a small illustration of the general observation that some functions are basic specifications and are themselves not subject to further specification. Because such basic specifications are generally very inefficient, they are mostly used in other specifications (that is, pre- or post-conditions,

loop invariants, general assertions) rather than in code intended to be executed.

2 Sorted Arrays

A number of algorithms on arrays would like to assume that they are sorted. Such algorithms would return a correct result only if they are actually running on a sorted array. Thus, the first thing we need to figure out is how to specify sortedness in function specifications. The specification function `is_sorted(A, lo, hi)` traverses the array A from left to right, starting at lo and stopping just before reaching hi , checking that each element is smaller than or equal to its right neighbor. We need to be careful about the loop invariant to guarantee that there will be no attempt to access a memory element out of bounds.

```

1 bool is_sorted(int[] A, int lo, int hi)
2 //@requires 0 <= lo && lo <= hi && hi <= \length(A);
3 {
4   for (int i = lo; i < hi-1; i++)
5     //@loop_invariant lo <= i;
6     if (!(A[i] <= A[i+1])) return false;
7   return true;
8 }
```

The loop invariant here does not have an upper bound on i . Fortunately, when we are inside the loop, we know the loop condition is true so we know $i < hi - 1$. That together with $lo \leq i$ guarantees that *both* accesses are in bounds.

We could also try $i \leq hi - 1$ as a loop invariant, but this turns out to be false. It is instructive to think about why. If you cannot think of a good reason, try to prove it carefully. Your proof should fail somewhere.

Actually, the attempted proof already fails at the initial step. If $lo = hi = 0$ (which is permitted by the precondition) then it is *not* true that $0 = lo = i \leq hi - 1 = 0 - 1 = -1$. We could say $i \leq hi$, but that wouldn't seem to serve any particular purpose here since the array accesses are already safe.

Let's reason through that. Why is the access $A[i]$ safe? By the loop invariant $lo \leq i$ and the precondition $0 \leq lo$ we have $0 \leq i$, which is the first part of safety. Secondly, we have $i < hi - 1$ (by the loop condition, since we are in the body of the loop) and $hi \leq \text{length}(A)$ (by the precondition), so i will be in bounds. In fact, even $i + 1$ will be in bounds, since $0 \leq lo \leq i < i + 1$ (since i is bounded from above) and $i + 1 < (hi - 1) + 1 = hi \leq \text{length}(A)$.

Whenever you see an array access, you must have a very good reason why the access must be in bounds. You should develop a coding instinct where you *deliberately pause* every time you access an array in your code and verify that it should be safe according to your knowledge at that point in the program. This knowledge can be embedded in preconditions, loop invariants, or assertions that you have verified.

3 Linear Search in a Sorted Array

Next, we want to search for an element x in an array A which we know is sorted in ascending order. We want to return -1 if x is not in the array and the index of the element if it is.

The pre- and post-condition as well as a first version of the function itself are relatively easy to write.

```
1 int search(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A,0,n);
4 /*@ensures (\result == -1 && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 {
8   for (int i = 0; i < n; i++)
9     //@loop_invariant 0 <= i && i <= n;
10    if (A[i] == x) return i;
11  return -1;
12 }
```

This does not exploit that the array is sorted. We would like to exit the loop and return -1 as soon as we find that $A[i] > x$. If we haven't found x already, we will not find it subsequently since all elements to the right of i will be greater or equal to $A[i]$ and therefore strictly greater than x . But we have to be careful: the following program has a bug.

```
1 int search(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A,0,n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 @*/
7 {
8   for (int i = 0; A[i] <= x && i < n; i++)
9     //@loop_invariant 0 <= i && i <= n;
10    if (A[i] == x) return i;
11  return -1;
12 }
```

Can you spot the problem? If you cannot spot it immediately, reason through the loop invariant. Read on if you are confident in your answer.

The problem is that the loop invariant only guarantees that $0 \leq i \leq n$ before the exit condition is tested. So it is possible that $i = n$ and the test $A[i] \leq x$ will try to access an array element out of bounds: the n elements of A are numbered from 0 to $n - 1$.

We can solve this problem by taking advantage of the so-called *short-circuiting evaluation* of the boolean operators of conjunction (“and”) `&&` and disjunction (“or”) `||`. If we have condition $e_1 \ \&\& \ e_2$ (e_1 and e_2) then we do not attempt to evaluate e_2 if e_1 is false. This is because a conjunction will always be false when the first conjunct is false, so the work would be redundant.

Similarly, in a disjunction $e_1 \ || \ e_2$ (e_1 or e_2) we do not evaluate e_2 if e_1 is true. This is because a disjunction will always be true when the first disjunct is true, so the work would be redundant.

In our linear search program, we just swap the two conjuncts in the exit test.

```

1 int search(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A,0,n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6 */
7 {
8   for (int i = 0; i < n && A[i] <= x; i++)
9     //@loop_invariant 0 <= i && i <= n;
10    if (A[i] == x) return i;
11  return -1;
12 }
```

Now $A[i] \leq x$ will only be evaluated if $i < n$ and the access will be in bounds since we also know $0 \leq i$ from the loop invariant.

Alternatively, and perhaps easier to read, we can move the test into the loop body.

```
1 int search(int x, int[] A, int n)
2 //@requires 0 <= n && n <= \length(A);
3 //@requires is_sorted(A,0,n);
4 /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6   @*/
7 {
8   for (int i = 0; i < n; i++)
9     //@loop_invariant 0 <= i && i <= n;
10    {
11      if (A[i] == x) return i;
12      else if (A[i] > x) return -1;
13    }
14   return -1;
15 }
```

This program is not yet satisfactory, because the loop invariant does not have enough information to prove the post-condition. We *do* know that if we return directly from inside the loop, then $A[i] = x$ and therefore $A[\text{\result}] == x$ holds. But we cannot deduce that $\text{!is_in}(x, A, 0, n)$ if we return -1.

Before you read on, consider which loop invariant you might add to guarantee that. Try to reason why the fact that the exit condition must be false and the loop invariant true is enough information to know that $\text{!is_in}(x, A, 0, n)$ holds.

Did you try to exploit that the array is sorted? If not, then your invariant is most likely too weak, because the function is incorrect if the array is not sorted!

What we want to say is that *all elements in A to the left of index i are smaller than x* . Just saying $A[i-1] < x$ isn't quite right, because when the loop is entered the first time we have $i = 0$ and we would try to access $A[-1]$. We again exploit short-circuiting evaluation, this time for disjunction.

```

1 int search(int x, int[] A, int n)
2   //@requires 0 <= n && n <= \length(A);
3   //@requires is_sorted(A,0,n);
4   /*@ensures (-1 == \result && !is_in(x, A, 0, n))
5           || ((0 <= \result && \result < n) && A[\result] == x);
6   @*/
7   {
8     for (int i = 0; i < n; i++)
9       //@loop_invariant 0 <= i && i <= n;
10      //@loop_invariant i == 0 || A[i-1] < x;
11      {
12        if (A[i] == x) return i;
13        else if (A[i] > x) return -1;
14        //@assert A[i] < x;
15      }
16    return -1;
17  }

```

It is easy to see that this invariant is preserved. Upon loop entry, $i = 0$. Before we test the exit condition, we just incremented i . We did not return while inside the loop, so $A[i-1] \neq x$ and also $A[i-1] \leq x$. From these two together we have $A[i-1] < x$. We have added a corresponding assertion to the program to highlight the importance of that fact.

Why does the loop invariant imply the post-condition of the function? If we exit the loop normally, then the loop condition must be false. So $i \geq n$. We know $A[n-1] = A[i-1] < x$. Since the array is sorted, all elements from 0 to $n-1$ are less or equal to $A[n-1]$ and so also strictly less than x and x cannot be in the array.

If we exit from the loop because $A[i] > x$, we also know that $A[i-1] < x$ so x cannot be in the array since it is sorted.

4 Analyzing the Number of Operations

In the worst case, linear search goes around the loop n times, where n is the given bound. On each iteration except the last, we perform three compar-

isons: $i < n$, $A[i] = x$ and $A[i] > x$. Therefore, the number of comparisons is almost exactly $3 \times n$ in the worst case. We can express this by saying that the running time is *linear* in the size of the input (n). This allows us to predict the running time pretty accurately. We run it for some reasonably large n and measure its time. Doubling the size of the input $n' = 2 \times n$ means that now we perform $3 \times n' = 3 \times 2 \times n = 2 \times (3 \times n)$ operations, twice as many as for n inputs.

We will introduce more abstract measurements for the running times in the next lecture.

Exercises

Exercise 1 (sample solution on page 13). Given an unsorted, non-empty array of integers A , the following code returns the index of the maximum element:

```

1 int find_max(int[] A, int n)
2 //@requires n == \length(A) && n > 0;
3 {
4     int max_index = 0;
5     int max_num = A[0];
6
7     for (int i = 1; i < n; i++)
8         //@loop_invariant 1 <= i && i <= n;
9     {
10         if (A[i] > max_num) {
11             max_index = i;
12             max_num = A[i];
13         }
14     }
15     return max_index;
16 }
```

1. Give the line number(s) which guarantee that line 5 is safe.
2. Give the line number(s) which guarantee that line 10 is safe.
3. Assume the 4-element array A contains the integer 5 in each position (pictorially, A is $[5, 5, 5, 5]$). What will `find_max(A, 4)` return?
4. Assume the 5-element array B contains the integers 1, 3, 2, 3 and 1 in this order (pictorially, B is $[1, 3, 2, 3, 1]$). What will `find_max(B, 5)` return?
5. This function has no postconditions, and therefore it is trivially correct.
 - Add a postcondition that allows the caller to use the value returned by this function safely.
 - Describe (without necessarily writing it in code) a postcondition that assures that this function achieves the desired outcome.

Exercise 2 (sample solution on page 13). Consider the following function

```

1 void doubling(int[] A, int[] B, int n)
2 //@requires n*2 <= int_max();
3 //@requires \length(A) == n && \length(B) == 2*n;
```

```

4 {
5   for (int i = 0; i < n; i++)
6     //@loop_invariant 0 <= i && i <= n;
7     {
8       B[2*i] = A[i];
9       B[2*i + 1] = A[i];
10    }
11 }

```

1. The precondition on line 2 may not be as useful as it appears at first sight. Explain why. Rewrite this precondition so that it actually achieves the intended constraints on the value of n .
2. Use your upgraded precondition to prove the safety of the array access $B[2*i+1]$ on line 9.

Exercise 3 (sample solution on page 14). Consider the following variant of linear search that looks for x in the portion of the array A between indices lo included and hi excluded:

```

1 int search(int x, int[] A, int lo, int hi)
2   //@requires 0 <= lo && lo <= hi && hi <= \length(A);
3   /*@ensures (\result == -1 && !is_in(x, A, lo, hi))
4           || (lo <= \result && \result < hi && A[\result] == x);
5   @*/
6   {
7     for (int i = lo; i < hi; i++)
8       //@loop_invariant lo <= i && i <= hi;
9       //@loop_invariant !is_in(x, A, lo, i);
10      {
11        if (A[i] == x)
12          return i;
13      }
14    //@assert !is_in(x, A, lo, hi);
15    return -1;
16 }

```

1. When we return from the function using the statement on line 12, we have to prove the second part of the postcondition: $(lo \leq \text{result} \ \&\& \ \text{result} < hi \ \&\& \ A[\text{result}] == x)$. Prove that this postcondition is correct.

2. When we return from the function using the statement on line 15, we know that `\result == -1`, so we have to prove the first part of the postcondition: `(\result == -1 && !is_in(x, A, lo, hi))`. Based on the way we wrote our code, this is trivial: we can point to the assertion on line 14, which tells us exactly what we need to know: `!is_in(x, A, lo, hi)`. We know that this assertion is safe because of the precondition to search, which tells us that $0 \leq lo \leq hi \leq \text{length}(A)$ (in math notation). Prove that the assertion will always hold true.

Sample Solutions

Solution of exercise 1

1. In order to know if the array access on line 5 is safe, we need to know that there is at least one element in the array. This is given by line 2.
2. We need to know that i is non-negative and less than the length of the array in order to prove that the array access on line 10 is safe. Line 8 guarantees that i is non-negative, and line 7 guarantees that it is less than n . However, we also need to know that n is actually the length of the array, which is given by line 2. So we need lines 2, 7 and 8.
3. `find_max(A,4)` returns 0 since no value in A is larger than what's in $A[0]$. This function returns the first index where the maximum occurs.
4. `find_max(B,5)` returns 1.
5. We can provision this function with the following postconditions.

- Since this function returns an array index, the caller will often access the array A at this index. The desired postcondition is therefore:

```
//@ensures 0 <= \result && \result < \length(A);
```

- The desired outcome is that the value at the returned index is greater than or equal to every value in A . We will learn in the next lecture about the specification function `ge_seg(x,A,lo,hi)`, which returns `true` if the value x is greater than or equal to every value in A between indices lo included and hi excluded. With such a function, the requested postcondition is therefore

```
//@ensures ge_seg(\result, A, 0, \length(A));
```

For an extra challenge, implement `ge_seg(x,A,lo,hi)`, or check it out in the next lecture.

Solution of exercise 2

1. The intention behind this precondition is for the function to be able to allocate the output array, as no array can have more than `int_max()` elements. However, every C0 `int` is less than or equal to `int_max()`. Therefore, line 2 is always true.

We can express the intended condition by constraining the input array instead. For the output array to have no more than `int_max()` elements, the input array should have at most `int_max() / 2` elements. Thus, we want to replace the given precondition with:

```
//@requires n <= int_max() / 2;
```

2. The proof goes as follows:

- | | |
|--|-----------------------------|
| a. $i < n$ | by line 5 |
| b. $i \leq n-1$ | by math on (a) |
| c. $2*(n-1) + 1 \leq \text{int_max}()$ | by the updated precondition |
| d. $2*i + 1 \leq 2*(n-1) + 1$ | by math on (b) and (c) |
| e. $\leq 2*n - 1$ | by math on (d) |
| f. $< 2*n$ | by math on (e) |
| g. $\text{\textcolor{brown}{length}}(B) == 2*n$ | by line 3 |
| h. $2*i + 1 < \text{\textcolor{brown}{length}}(B)$ | by math on (f) and (g) |
| i. $0 \leq i$ | by line 6 |
| j. $0 \leq 2i+1$ | by math on (i) and (a) |

Solution of exercise 3

- | | |
|--|------------------------|
| 1. a. $lo \leq i$ | by line 8 |
| b. $\text{\textcolor{brown}{result}} == i$ | by line 12 |
| c. $lo \leq \text{\textcolor{brown}{result}}$ | by math on (a) and (b) |
| d. $i < hi$ | by line 7 |
| e. $\text{\textcolor{brown}{result}} < hi$ | by math on (b) and (c) |
| f. $A[i] == x$ | by line 11 |
| g. $A[\text{\textcolor{brown}{result}}] == x$ | by math on (f) and (b) |
| h. $(lo \leq \text{\textcolor{brown}{result}} \ \&\& \ \text{\textcolor{brown}{result}} < hi \ \&\& \ A[\text{\textcolor{brown}{result}}] == x)$ | by (c), (e) and (g) |
| 2. a. $i \leq hi$ | by line 8 |
| b. $i \geq hi$ | by line 7 |
| c. $i == hi$ | by math on (a) and (b) |
| d. $!is_in(x, A, lo, i)$ | by line 9 |
| e. $!is_in(x, A, lo, hi)$ | by math on (c) and (d) |